

第 7 章 出错了怎么办

第零章见过几种报错，但那时只是看看。这一章正面处理错误，让程序遇到意外时不至于整个崩掉，也学会怎么把藏起来的问题找出来。

7.1 先做一个不会崩的计算器

本节任务：写一个把字符串转成数字再相除的函数，用户输入乱七八糟的东西它也不崩，而是给一句人话提示。

```
1 def divide(a, b):
2     """把两个字符串转成数字相除，出问题就返回一句说明"""
3     try:
4         x = float(a)
5         y = float(b)
6         return x / y
7     except ValueError:
8         return "输入的不是数字"
9     except ZeroDivisionError:
10        return "除数不能是零"
11
12 print(divide("10", "4")) # 2.5
13 print(divide("十", "4")) # 输入的不是数字
14 print(divide("10", "0")) # 除数不能是零
15 print(divide("10", "2.5")) # 4.0
```

运行结果：

```
2.5
输入的不是数字
除数不能是零
4.0
```

如果不写 try，第二次调用就会让整个程序停下，后面两行根本不会执行。这就 ★核心是处理错误的意义，不是让错误消失，而是让程序在出错时仍能有个交代、继续往下走。

7.2 try 和 except

本节任务：会捕获错误，并且只捕获自己预料到的那种。

基本结构是 try 里面放可能出错的代码，except 里面放出错后的处理：

```
1 try:
2     n = int("abc")
3 except ValueError:
4     print("转换失败")
```

可以写多个 `except` 分别处理不同的错误，像第一节那样。也可以在一个 `except` 里写多种：

```
1 try:
2     ...
3 except (ValueError, TypeError):
4     print("要么类型不对，要么值不对")
```

想知道错误的具体信息，用 `as` 接住它：

```
1 try:
2     int("abc")
3 except ValueError as e:
4     print("出错了：", e) # 出错了： invalid literal for int() with base 10:
                          'abc'
```

有一个写法要避免。不要用光秃秃的 `except` 接住所有错误：

★ 核心

```
1 try:
2     result = compute()
3 except: # 不好，什么都接住
4     print("出错了")
```

这样写会把所有问题都吞掉，包括你自己写错代码导致的拼写错误。结果是程序不崩了，但行为不对，还查不出原因。正确做法是明确写出预料到的错误类型，没预料到的就让它崩出来，崩掉反而能第一时间发现问题。

完整的结构还有两块：

```
1 try:
2     f = open("data.txt", encoding="utf-8")
3 except FileNotFoundError:
4     print("文件不存在")
5 else:
6     print("读到了", len(f.read()), "个字符") # 没出错才执行
7 finally:
8     print("这句无论如何都会执行") # 常用来做清理
```

7.3 常见错误对照

本节任务：认全最常撞见的几种错误，看到名字就知道大概哪里出了问题。

表 7.1：最常见的几种错误。

错误	通常是什么原因
SyntaxError	语法写错了，括号引号不成对或少了冒号
NameError	用了不存在的名字，多半拼错或忘了加引号
TypeError	类型不对，比如字符串直接加数字
ValueError	类型对但值不合适，比如把汉字转成数字
IndexError	列表下标超出范围
KeyError	字典里没有这个键
ZeroDivisionError	除数是零
FileNotFoundError	要打开的文件不存在
AttributeError	对象没有这个属性或方法，常见于拼错方法名

分清 `TypeError` 和 `ValueError` 对新手很有用。类型不对是 `TypeError`，类型对但值不行是 `ValueError`。比如 `int("abc")` 报的是 `ValueError`，因为传的确实是字符串，只是这个字符串没法变成数字。

7.4 主动抛出错误

本节任务：在参数不合理时，主动报错而不是返回一个奇怪的值。

写函数时如果发现输入不合理，与其硬着头皮算下去，不如直接报错。用 `raise`：

```
1 def set_age(age):
2     if age < 0:
3         raise ValueError("年龄不能是负数")
4     return age
5
6 set_age(-5) # ValueError: 年龄不能是负数
```

这样调用方能立刻知道哪里错了，而不是拿着一个错误的结果继续跑。选哪种错误类型有个简单标准，值不合理用 `ValueError`，类型不对用 `TypeError`。

7.5 怎么把问题找出来

本节任务：程序没报错但结果不对时，用几个笨办法定位问题。

比报错更难缠的是不报错但结果不对。有几个朴素但有效的办法。

第一个是**打印中间结果**。怀疑哪一步就在哪一步打印出来看：

```
1 def average(nums):
2     total = sum(nums)
3     print("调试: total =", total, "个数 =", len(nums)) # 查完记得删掉
4     return total / len(nums)
```

这是最土也最常用的办法，别嫌笨。

第二个是**断言**。用 `assert` 写下你认为必然成立的条件，不成立就立刻报错：

```
1 def average(nums):
2     assert len(nums) > 0, "列表不能是空的"
3     return sum(nums) / len(nums)
4
5 average([]) # AssertionError: 列表不能是空的
```

断言的好处是把问题暴露在源头。没有它的话，空列表要到除法那一步才报 `ZeroDivisionError`，离真正的原因已经隔了一层。

第三个是**缩小范围**。程序很长时，把可疑的那几行单独拎出来，用最小的数据跑一遍。多数问题在这一步就现形了。

第四个是仔细读报错的最后几行。它会指出出错的文件和行号，顺着往上还能看到是从哪里调用过来的。这条调用链常常比错误本身更能说明问题。

7.6 小结

这一章让程序面对意外时更从容。`try` 里放可能出错的代码，`except` 处理预料到的错误，用 `as` 能拿到错误详情，`else` 在没出错时执行，`finally` 无论如何都执行。不要用光秃秃的 `except` 接住一切，那会把自己的 `bug` 也吞掉。分清类型不对的 `TypeError` 和值不合适的 `ValueError`。参数不合理时用 `raise` 主动报错。查问题的四个办法是打印中间结果、写断言、缩小范围、读报错的调用链。下一章学怎么用类把数据和操作打包在一起。

本章知识点

- **try except** —— `try` 里放可能出错的代码，`except` 处理预料到的错误
- **as e** —— 接住错误对象，可以打印出具体原因
- 光秃秃的 **except** —— 接住所有错误的写法，会把自己的 `bug` 也吞掉，不要用
- **else** 与 **finally** —— `else` 在没出错时执行，`finally` 无论如何都执行，常用来清理
- **TypeError** 与 **ValueError** —— 类型不对是前者，类型对但值不合适是后者

- **raise** —— 主动抛出错误，参数不合理时比返回怪值更好
- **打印中间结果** —— 最土也最常用的排查办法
- **assert** —— 断言必然成立的条件，把问题暴露在源头
- **缩小范围** —— 把可疑的几行单独拎出来用最小数据跑
- **调用链** —— 报错最后几行指出文件行号和调用来源，常比错误本身更有用